FSD - Harnessing Gen AI: From Design to Code Optimization

Generative AI in Code Migration and Optimization

Engage and Think

Imagine you are a software development team member involved in the CRM system optimization process and its migration to modern architecture. The goal is to achieve higher performance, remove duplication, and increase scalability. To meet deadlines efficiently, you suggested integrating Generative AI into the whole process.

How might Generative AI facilitate the optimization and migration of the CRM system, ensuring smoother transitions, improved efficiency, and adherence to project timelines?

Learning Objectives

By the end of this lesson, you will be able to:

- Utilize Generative AI tools for code migration to streamline the process and reduce manual effort
- Demonstrate the use of Generative AI in optimizing code by providing practical examples and evaluating the improvements
- Implement steps to enhance code performance by focusing on time complexity using Generative AI tools to streamline code execution
- Execute steps for optimizing code regarding space complexity using Generative AI tools to reduce memory usage and improve efficiency

Code Migration and Debugging: Streamlining with Generative AI

Code Migration

It is the process of converting code from one programming language, framework, or tech stack to another while preserving its functionality.

It is generally done to modernize systems, improve performance, enhance compatibility, or transition to more efficient platforms or technologies.

Types of Code Migration

Generative AI helps in the following code migration tasks, enhancing efficiency and reducing errors in the process:

01

Language migration

02

Framework migration

03

Version upgrade

Generative AI in Code Migration: Advantages

It expedites code migration by automating the conversion process. The following are key advantages of using Generative AI for code migration:

Automation

Simplifies the conversion process, minimizing manual intervention

Accuracy

Maintains accuracy in both syntax and functionality throughout the migration

Speed

Accelerates the migration process, outpacing traditional methods

Consistency

Ensures consistency in code structure, mitigating the potential for human error

Generative AI Tools for Code Migration

Utilize OpenAI Codex to swiftly convert code between programming languages with advanced AI assistance

Optimize programming language conversion using Tabnine's AI-driven code completion capabilities

Improve code security and quality with Snyk's AI-driven static code analysis. Snyk uses DeepCode AI to identify vulnerabilities and suggest fixes in real time

Generative AI Tools for Code Migration

Generate refactored code in the target language or framework using GitHub Copilot's AI-powered suggestions and context-based completions

Perform code migration using ChatGPT's AI-driven insights and personalized guidance to adapt and refactor code for new environments

Language Conversion with Generative AI

It is the process of translating code from one programming language to another while maintaining its original functionality and logic.

Common scenarios include migrating legacy systems to modern languages and frameworks, such as integrating React with PHP to enhance front-end capabilities while maintaining existing backend logic.

Steps in Language Conversion without Generative AI

Code analysis	Examine and comprehend the structure of the existing code
Mapping	Align elements from the original language to the target language
Conversion	Translate the code's syntax and logic to the target language
Testing	Verify that the functionality remains unchanged after the conversion
Optimization	Enhance the performance and readability of the newly converted code

Steps in Language Conversion Using Generative AI

Input source code

Provide the existing or original code to the generative AI tool

Select target language

Choose the programming language to convert the code

Generate prompt

Develop a clear and concise prompt for generative AI

Generate code

Allow the AI tool to convert the source code into the equivalent target language

Debug

Identify and fix errors to ensure the code functions correctly

Demo: Facilitating Language Conversion Using Generative AI

Duration: 10 minutes

Demonstrate how to facilitate language conversion using generative AI

You are overseeing a project aimed at modernizing the login functionality of an online retail platform by converting it from Java to JavaScript.

This demonstration highlights how GitHub Copilot facilitates the conversion of Java code that interacts with a MySQL database to JavaScript using Node.js. This approach enhances the platform's adaptability and efficiency in handling user authentication and data management.

DEMONSTRATION

Note: Refer to the demo document for detailed steps:
Demo_01_Facilitating _Language_Conversion_Using_Generative_AI

Framework Migration

It is the process of converting an application from one software framework to another while ensuring that its core functionality and overall architecture remain intact.

It is done to keep up with evolving technology trends, improve system performance, and maintain a competitive edge in software development.

Steps for Framework Migration Using Generative AI

For example, if there is a requirement to migrate the codebase from Django to the Flask framework, the following steps outline how to achieve this using generative AI:

Input source framework code

Provide the existing Django code to the generative AI tool for analysis

Select target framework

Select Flask as the target framework for migration

Generate code

Allow the AI tool to process the Django code and generate equivalent Flask code

Debug and refine

Review, debug, and optimize the generated Flask code, as needed

Debugging with Generative AI

It is the process of identifying, analyzing, and correcting bugs in software.

Incorporating generative AI can accelerate this process by automatically detecting errors and suggesting fixes.

How Generative AI Transforms Debugging

01

Automating error detection
Quickly scans code to identify syntax and logical errors

02

Suggesting fixes
Offers code corrections and optimizes existing implementations

03

Learning from interactions
Improves its suggestions based on feedback and past corrections

Generative AI Tools for Debugging

For example, if there is a requirement to migrate the codebase from Django to the Flask framework, the following steps outline how to achieve this using generative AI:

GitHub copilot:

Provides context-aware code suggestions.

AI-powered IDE Plugins

Enhances traditional development environments with AI capabilities

Automated Testing Frameworks

Utilizes AI to generate tests that cover the edge cases often missed by manual testing

Demo: Debugging a React.js Application

Duration: 10 minutes

Demonstrate setting up and debugging a React.js e-commerce application using the Vite framework

You are tasked with developing key frontend components within a single-page application architecture to ensure smooth user interactions and effective state management.

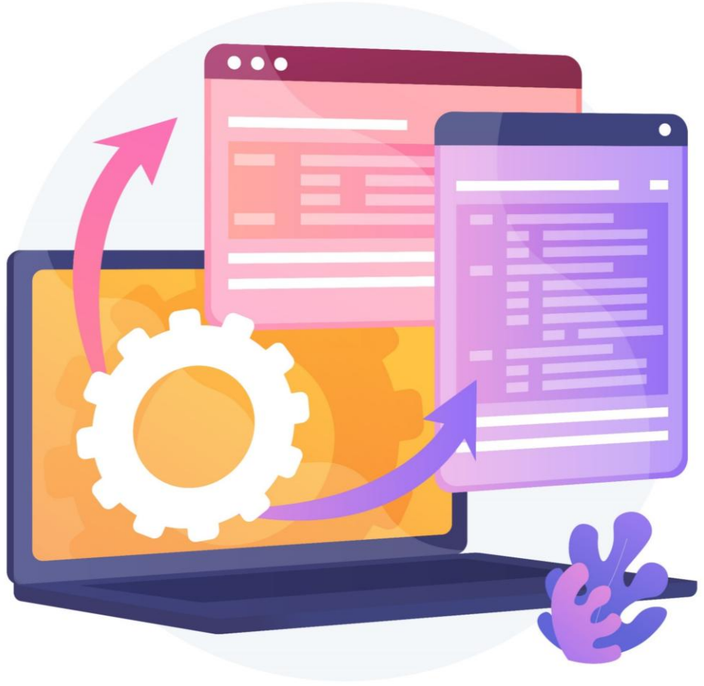
This demo will walk you through the initial setup with Vite, focusing on implementing essential features like product display, shopping cart functionality, and navigation. You'll also set up routing and manage state across the application. The session wraps up with debugging techniques using browser development tools and React Developer Tools to ensure the application runs smoothly and efficiently.

DEMONSTRATION

Note: Refer to the demo document for detailed steps:
Demo_02_Debugging_a_React.js_Application

Version Upgrades

It involves transitioning software or frameworks to newer versions to stay current with advancements and improvements.



It ensures access to new features, enhanced security measures, and improved performance, vital for maintaining the effectiveness and relevance of the software or framework.

Steps in Version Upgrades without Generative AI

Review release notes
Review the new features and changes

Dependency management
Align dependencies with compatible versions

Code modification
Modify code to meet new requirements

Testing
Conduct comprehensive testing for issues

Deployment
Deploy the upgraded version effectively

Steps in Version Upgrades Using Generative AI

Analyze current code

Input the current code into the AI platform for thorough analysis

Specify target version

Specify the desired version to upgrade the software

Generate upgraded code

AI automatically adjusts the code for compatibility with the selected version

Review and refine

Perform manual review and optimization of the upgraded software

Version Upgrades: Example

Generative AI can automate version upgrading by analyzing patterns in code from an older version and generating equivalent code for a newer version.

Example

The conversion of a login system functionality code from Python 2.7 to 3.6 is a manual update process that is time-consuming and prone to errors, which can lead to significant delays and potential compatibility issues.

By leveraging Generative AI, outdated Python 2.7 syntax can be identified
and replaced with the optimized
Python 3.6 equivalent.

Sure! Below is the converted code for Python 3.6 with the changes highlighted.

python Copy code

```python
import getpass

# Sample dictionary of usernames and passwords
users = {
    'user1': 'password1',
    'user2': 'password2',
    'user3': 'password3'
}

def login():
    # Changes from raw_input to input for Python 3.6
    username = input("Enter username: ")
    password = getpass.getpass("Enter password: ")

    if username in users and users[username] == password:
        print("Login successful!")
    else:
        print("Login failed. Invalid username or password.")

if __name__ == "__main__":
    login()
```

Changes Highlighted:

1. raw_input to input:
   - In Python 2.7, `raw_input` is used to get user input as a string.
   - In Python 3.x, `raw_input` was renamed to `input`.

Use Cases of Using Generative Ai in Code Optimization

Use Case: Uber's Code Migration and Refactoring

Problem

Uber needed to migrate and refactor large codebases, which is time-consuming and error-prone.

Solution

Uber leveraged Generative AI to build a prototype to automate code refactoring and centralized migrations. This approach helped identify migration patterns and refactor large codebases more efficiently, allowing platform teams to save engineering time and focus on other critical tasks.

Uber

Quick Check

You are migrating a legacy customer management system from Java to JavaScript to enhance its front-end capabilities and improve user interaction. What action should you first take when using Generative AI tools for this migration?

A. Update the dependencies manually

B. Review the new features of JavaScript

C. Input the existing Java code into the AI tool

D. Begin testing the new JavaScript code

Optimizing Codebase Using Generative AI

Code Optimization

It involves refining code to enhance efficiency, reducing execution time and resource consumption while preserving functionality.

The goal of code optimization is to attain optimal performance, optimizing speed (time complexity), and resource usage (space complexity).

Process of Code Optimization

It involves the following crucial steps to enhance performance and efficiency:

Profiling

Locates performance issues within the code

Process

Benchmarking

Compares performance metrics before and after optimization

Why Do We Need Code Optimization?

Code optimization is crucial to improve various aspects of software performance, including:

Time complexity	Space complexity	User experience	Cost efficiency
Improves code execution speed, which is essential for performance-sensitive applications	Optimizes memory, which is critical for apps on devices with limited resources	Enhances responsiveness and reliability, leading to better user satisfaction	Minimizes operational costs by reducing resource consumption

What Generative AI Does in Code Optimization?

Generative AI plays the following roles in optimizing an application's codebase:

Time optimization

Reduces execution time for faster processing

Space optimization

Minimizes memory usage for better resource management

How Generative AI Helps in Code Optimization?

Generative AI can significantly enhance code optimization through the following methods:

Automated refactoring

Enhances code efficiency through automatic refactoring processes

Complex analysis

Manages intricate optimization tasks that are laborious for developers

Pattern recognition

Detects inefficient coding patterns and recommends optimal solutions

Continuous improvement

Integrates with CI/CD pipelines to ensure ongoing code optimization

Generative AI Tools for Code Optimization

Enhances code optimization with real-time AI suggestions and auto-completions, improving development speed and quality

Codacy

Provides automated code analysis and quality monitoring, identifying areas for improvement and optimization in software projects

Optimizes code performance, identifying inefficiencies and providing recommendations for improvement in AWS-based applications

Quick Check

You are optimizing the performance of a logistics application that schedules deliveries. The current system is slow due to inefficient code. Which feature of Generative AI is most beneficial for optimizing this code?

A. Automate refactoring to enhance efficiency

B. Update to the latest version of the programming language

C. Add more hardware resources

D. Reduce the number of applications on the system

Enhancing Code Performance: Time Complexity

Understanding Time Complexity

It refers to the measure of the amount of time required for an algorithm to complete its execution as a function of the size of the input data.

Notation

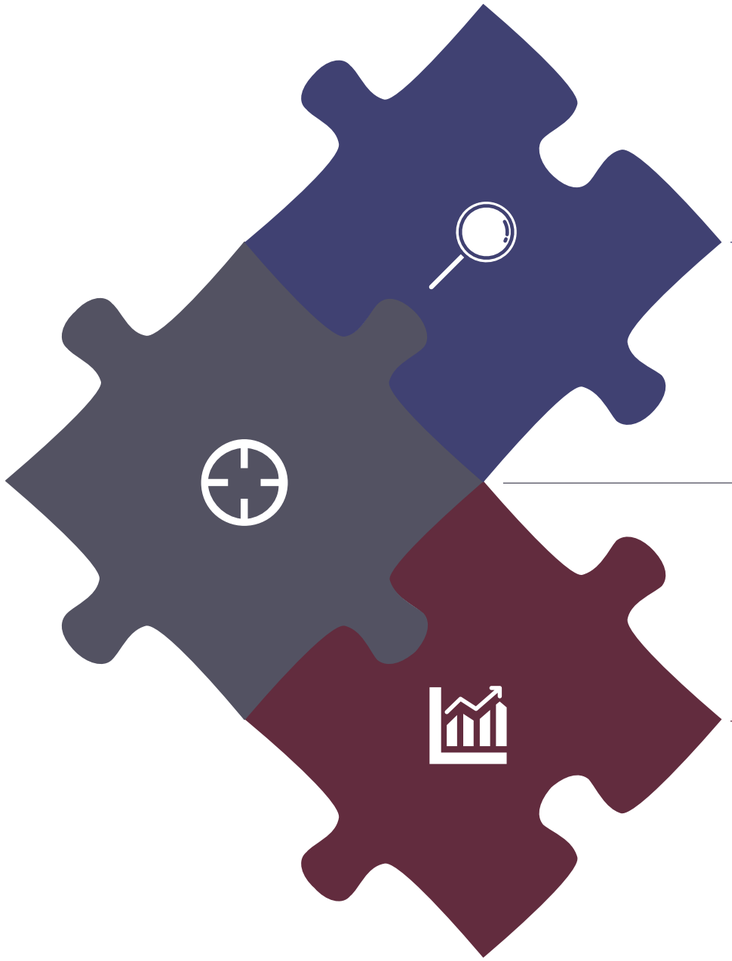
Time complexity is represented using Big O notation (e.g., O(n), O(log n)) for scalability analysis.

Impact

Time complexity directly influences application speed and responsiveness.

Optimize Time Complexity with Generative AI

Generative AI utilizes the following factors to assist in code optimization:



Analysis

Conducts code analysis to identify inefficient algorithms

Reviews

Recommends efficient algorithms and data structures to replace the inefficient ones

Refactoring

Refactors the codebase to minimize time complexity, improving overall efficiency

Time Complexity with Generative AI: Example

Generative AI can conduct in-depth analysis of algorithms to identify areas with high time complexity.

Problem

An optimized profit calculation algorithm is capable of handling large datasets efficiently. The current implementation iterates through all possible combinations of three sales, resulting in poor performance and increased processing time.

There is a critical need to refactor the algorithm to improve efficiency, minimize computational overhead, and enhance overall performance.

```python
def calculate_profit(sales):
    max_profit = 0

    # Iterate over all possible combinations of three sales
    # Time complexity: O(n^3)
    for i in range(len(sales)):
        for j in range(i+1, len(sales)):
            for k in range(j+1, len(sales)):
                profit = sales[i] + sales[j] + sales[k]
                max_profit = max(max_profit, profit)

    return max_profit

# Example usage
sales = [10, 20, 30, 40, 50]
max_profit = calculate_profit(sales)
print("Maximum profit:", max_profit)
```

Time Complexity with Generative AI: Example

Generative AI enhances efficiency in profit calculation through optimization techniques.

Solution

Generative AI tools enhance time complexity, resulting in improved execution speed within software applications.

The implementation showcases the efficacy of AI-driven optimizations, underscoring the potential for accelerated performance and efficiency gains.

```
def calculate_profit(sales):
```

Improve its time complexity

GitHub Copilot

Made changes.

Accept Discard

Accept or Discard 1 change.

```
    sales.sort()  # Sort the sales in ascending order
    max_profit = 0

    # Iterate over the last three sales
    # Time complexity: O(n)
    for i in range(len(sales)-3, len(sales)):
        max_profit += sales[i]

    return max_profit

# Example usage
sales = [10, 20, 30, 40, 50]
max_profit = calculate_profit(sales)
print("Maximum profit:", max_profit)
```

Demo: Analyzing and Optimizing Existing Code Using Generative AI

Duration: 10 minutes

Demonstrate how to analyze and optimize existing code using generative AI

You are tasked with enhancing the efficiency of the ExpressGo parcel delivery system's existing codebase, focusing on improving operations such as search, sort, and statistics. When using generative AI tools like GitHub Copilot, the goal is to refine the Python scripts to achieve better time complexity and performance.

The process ensures that ExpressGo's system maintains its functional integrity and operates more efficiently, demonstrating the practical benefits of using generative AI in software development.

DEMONSTRATION

Note: Refer to the demo document for detailed steps:
03_Analyzing_and_Optimizing_Existing_Code_Using_Generative_AI

Quick Check

You are working on an e-commerce platform where the search functionality is slow due to many nested loops in the code. How can Generative AI assist in improving the time complexity of the search functionality?

A. By conducting code profiling to identify memory usage

B. By suggesting alternative algorithms with lower time complexity

C. By converting the code to a different programming language

D. By integrating new hardware to speed up processing

Code Optimization: Space Complexity

Understanding Space Complexity

It quantifies the amount of memory an algorithm requires relative to the size of the input data.

Notation

Typically represented using Big O notation (e.g., O(n), O(1)) to assess scalability and memory usage

Impacts

Directly influences the memory consumption and scalability of an application

Optimize Space Complexity with Generative AI

Generative AI can assist in reducing space complexity through the following aspects:

Analysis

Conducts thorough analysis of the codebase to pinpoint sections with excessive memory

Review

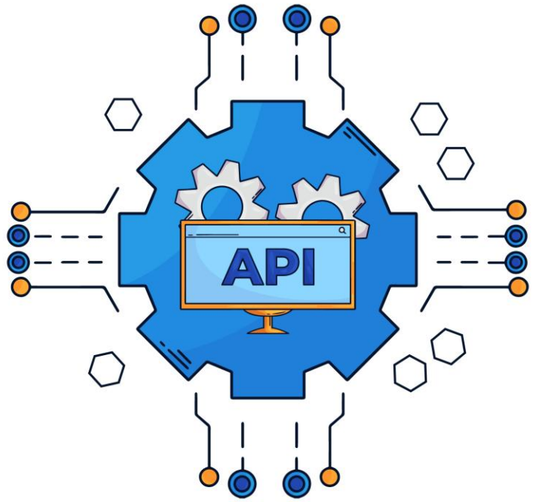
Suggests or implements techniques to minimize memory usage

Optimization

Utilizes various techniques such as in-place algorithms and efficient data structures

Optimizing Memory Usage Through API Design

APIs (Application Programming Interfaces) are sets of rules and specifications that allow different software applications to communicate with each other.



Efficient API design can significantly reduce space complexity by minimizing redundant data transfer and optimizing data structures used in application interactions.

Space Complexity with Generative AI: Example

Generative AI scrutinizes code to pinpoint sections with excessive memory usage, identifying areas for optimization.

Problem

Develop a solution to optimize an e-commerce store's functionality; the existing implementation of the function exhibits high space complexity. With three nested loops iterating over the product list, the scenario incurs a space complexity of O(n^3).

This inefficiency severely impacts scalability and performance, especially with large datasets.

```python
def calculate_total_price(products):
    """
    Calculates the total price of products in an ecommerce store.

    Args:
        products (list): A list of product prices.

    Returns:
        int: The total price of all products.

    Space Complexity:
        O(n^3) - The space complexity is O(n^3) because we have three
                 nested loops that iterate over the products list.
    """
    n = len(products)
    total_price = 0

    for i in range(n):
        for j in range(n):
            for k in range(n):
                total_price += products[i] * products[j] * products[k]

    return total_price
```

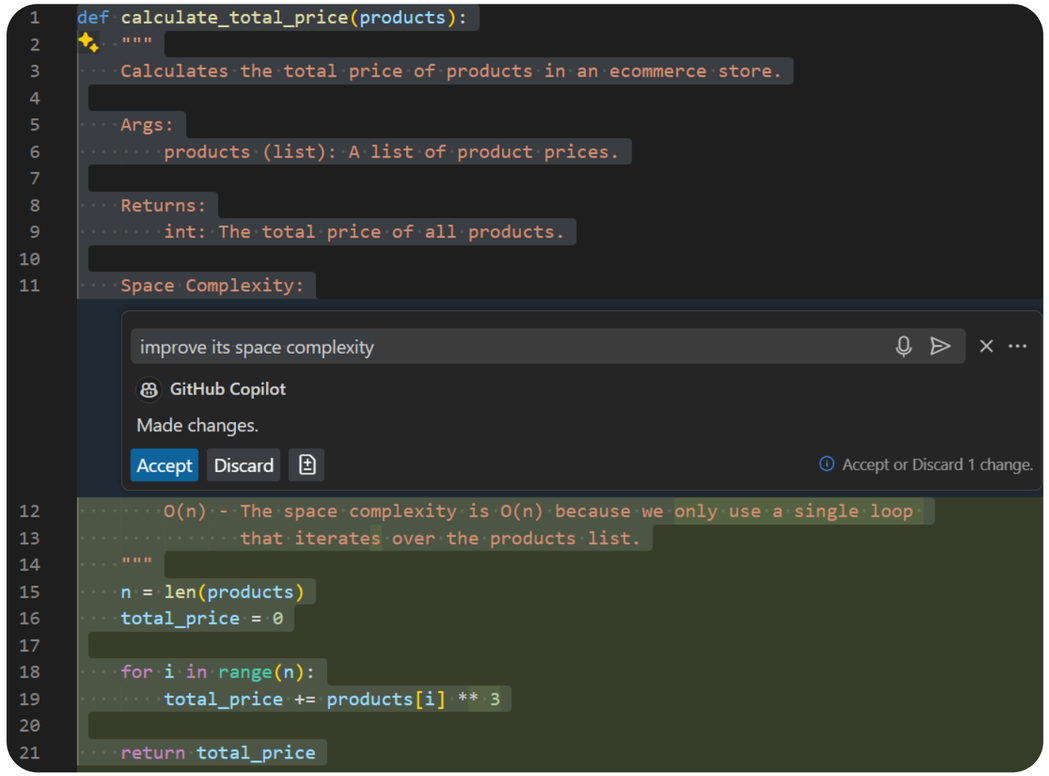
Space Complexity with Generative AI: Example

Generative AI employs compression methods to reduce the size of stored data, enhancing space efficiency.

Solution

A refined solution has been developed using Generative AI to address the space complexity issue. By optimizing the algorithm to utilize a single loop iterating over the products list, the space complexity has been reduced to O(n).

This enhancement ensures improved efficiency and scalability, making the function suitable for handling larger datasets within the e-commerce store.



Demo: Analyzing and Improving Space Complexity of Code Using Generative AI

Duration: 10 minutes

DEMONSTRATION

Demonstrate how to analyze and improve the space complexity of code using generative AI

You are tasked with optimizing the space complexity of APIs for ExpressGo, a parcel delivery service, focusing on enhancing memory usage efficiency. Utilize GitHub Copilot to analyze and refine, ensuring the application remains efficient and scalable.

The process involves setting up the project environment, integrating GitHub Copilot to generate optimized space complexity code for the search and sort functions, and testing these changes to confirm their effectiveness.

Note: Refer to the demo document for detailed steps:
04_Analyzing_and_Improving_Space_Complexity_of_Code_Using_Generative_AI

Quick Check

You manage a social media application that frequently runs out of memory due to inefficient data structures in user data management. Which method would Generative AI use to optimize the space complexity of your application?

A. Increase the server memory capacity

B. Analyze and recommend efficient data structures

C. Improve the network bandwidth

D. Optimize the user interface design

Guided Practice

Overview **Duration: minutes**

This project aims to analyze, improve, and migrate an existing retail management system's codebase to enhance its efficiency using generative AI. The task involves migrating from CherryPy to Node JS Express Framework and leveraging GitHub Copilot to streamline the transition. Furthermore, a comprehensive analysis of time and space complexity will identify opportunities for optimization. Ultimately, the project aims to optimize functionality, security, and compatibility within the retail management system, ensuring an enhanced user experience.

Tasks:

1. Set up Mongo DB
2. Analyze and execute the existing codebase
3. Evaluate the best project packages using Snyk Advisor
4. Migrate from CherryPy to Express Node JS Framework using GitHub Copilot

Practice Project

Migrate CherryPy to Express with Generative AI to Enhance Efficiency

This project aims to analyze, improve, and migrate an existing retail management system's codebase to enhance its efficiency using generative AI. The task involves migrating from CherryPy to Node.js Express Framework and leveraging GitHub Copilot to streamline the transition. Furthermore, a comprehensive analysis of time and space complexity will identify opportunities for optimization. Ultimately, the project aims to optimize functionality, security, and compatibility within the retail management system, ensuring an enhanced user experience.

Practice Project

Migrate CherryPy to Express with Generative AI to Enhance Efficiency

Task:

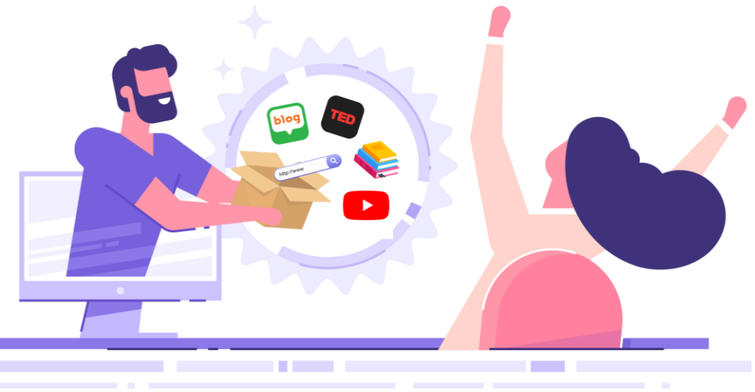
1. Set up MongoDB
2. Analyze and execute the existing codebase
3. Evaluate the best project packages using Snyk Advisor
4. Migrate from CherryPy to Express Node.js Framework using GitHub Copilot

PRACTICE PROJECT

Key Takeaways

- Generative AI automates code conversion across programming languages and frameworks, ensuring accuracy and consistency.
- It facilitates seamless framework migration and version upgrades by analyzing existing codebases and generating optimized code for target environments.
- AI tools refactor code, identify inefficient patterns, and implement algorithmic improvements for enhanced performance optimization.
- Generative AI mitigates time complexity issues by suggesting and refactoring code with more efficient algorithms and data structures.
- It optimizes space complexity by pinpointing memory bottlenecks and employing memory-efficient algorithms, data compression, and optimized data structures.

Additional Resources



- Refer to this link for Code Migration with Generative AI https://www.ibm.com/architectures/hybrid/genai-modernization-and-code-generation
- Refer to this link for GPT Migrate https://medium.com/@fareedkhandev/gpt-migrate-convert-your-codebase-to-any-language-framework-cc1f846e4630
- Refer to this link for Amazon CodeGuru https://aws.amazon.com/blogs/machine-learning/optimizing-application-performance-with-amazon-codeguru-profiler/
- Refer to this link for trends and Challenges http://www.ijtrd.com/papers/IJTRD5476.pdf

Q&A